

API V3 Documentation Updates from last document version

- **Clarification of Delete Operations:** The documentation now explicitly states that a `DELETE` operation in API V3 permanently erases the entity or field rather than setting it to null or applying default values.¹
- **Filter and Grouping Syntax:** Significant updates were made to the documentation regarding grouping operators and filter syntax, including the consolidation of logical operator definitions.¹
- **Field Returns and Expansion:** The `_return` parameter has been replaced by `_fields`. Additionally, new guidance notes that `_fields=*` and `GET` operations should return consistent results, and `_ids` fields are hidden to prevent exposure of denormalization.¹
- **Bulk and Async Job Endpoints:** The documentation for bulk and asynchronous actions (e.g., jobs, bulk additions, and custom actions) was reorganized and updated to clarify paths, methods, and response payloads.¹
- **Product Relationships:** The terminology for relationship filters was updated, changing instances of `relationship_links` to `product_relationships`.¹

Grouping Operators & Filters

- **Simplification of Columns:** The legacy `v1/2` column was completely removed from the Grouping Operators and Filters specification tables, leaving only the direct `Logic` to `V3` mappings.¹²
- **Removal of Alternative Formats:** Alternative filter syntaxes—such as `field[not]=value`, `_not[0]field=value`, `field[!!eq]=value`, and `field[~eq]=value`—were removed, standardizing strictly on the primary V3 formats like `field[!eq]=value`.¹²

Field Projections & Selection

- **Deprecation of `_return`:** The `_return` parameter (previously used for mandatory field selection, returning all fields via `*`, or `_all`) has been entirely removed from the first-level and multilevel fields specifications. It is replaced fully by the `_fields` parameter.¹²
- **Exclusion of `_ids` Fields:** A new rule specifies that `_ids` fields are now hidden and excluded from API responses and operational use to avoid exposing denormalization.²

- **Behavior Alignment:** Explicit alignment notes were added stating that `_fields=*` and a standard `GET` request must return identical results, and that `_fields=related_entity` behaves identically to `_fields=related_entity.*`.²

Response Conventions

- **Addition of DELETE Specifications:** A new section explicitly defines the behavior of `DELETE` operations in V3. A `DELETE` operation permanently erases the targeted entity or field (when using a path-based delete) rather than setting values to null or applying defaults, which may trigger inheritance resolution for that field.²
- **Removal of PUT Evaluation:** The discussion and evaluation regarding the risks and complexities of `PUT` operations (such as unintended overwrites and full resource replacement) were removed, focusing the documentation updates strictly on `PATCH` and `DELETE`.¹²

Color codes

Reserved words

Forbidden field or entity names

Grouping Operators

Functions

Filters

Related entities filtering transversal

Returned first level fields

Returned multilevel fields

Related entities data expansion

Pagination

URL S STANDARDIZATION

Response Conventions for Entity Modification Endpoints

General

PATCH

DELETE

ANEXO

PROCESS MANAGER API

URL S STANDARDIZATION

Color codes		
Approved direct option	Approved alternative	Postponed

Reserved words
Starting with _

Forbidden field or entity names
Starting with _ (the only exception are attribute names, because there is no API endpoint that goes directly to that entity, and therefore, it always has the entity name first, being unambiguous. ie. <code>attributes._<attribute_name></code>)
Containing a .
Containing a [

Grouping Operators	
Logic	V3
<filter1> AND <filter2>	<filter1>&<filter2> _and[0]<filter1>&_and[0]<filter2>
<filter1> OR <filter2>	_or[#]<filter1>&_or[#]<filter2>
(<filter1> OR <filter2>) AND (<filter3> OR <filter4>)	_or[0]<filter1>&_or[0]<filter2>& _or[1]<filter3>&_or[1]<filter4> _and[0]_or[0]<filter1>&_and[0]_or[0]<filter2>& _and[0]_or[1]<filter3>&_and[0]_or[1]<filter4>
(<filter1> AND <filter2>) OR (<filter3> AND <filter4>)	_or[0]_and[0]<filter1>&_or[0]_and[0]<filter2>& _or[0]_and[1]<filter3>&_or[0]_and[1]<filter4>
NOT (<filter1> <bool> <filter2>)	<!bool>[0]<filter1>&<!bool>[0]<filter2>

Grouping Operators	
	eg: <code>_!or[0]<filter1>&_!or[0]<filter2></code>
<pre> SELECT ... FROM t1 LEFT JOIN t2 ON t1.t2_fk = t2.id AND <t2_filter> WHERE t2.id IS NULL AND EXISTS (SELECT * FROM T2 WHERE) </pre>	<code>_!exists[#]<t2_filter></code> For public APIs, this must be heavily restricted. For the public PIM API V3.0, it will only be allowed for <code>relationships.id</code> or <code>relationships.label</code> and without being grouped with <code>_and</code> or <code>_or</code>
<pre> SELECT ... FROM t1 LEFT JOIN t2 ON t1.id = t2.t1_fk AND <t2_filter> WHERE t2.id IS NULL </pre>	
(3+ level nesting)	

Functions	
General pattern (<field>[<function>] The 1st arg is the value of the field) (<entity_x:n>[<function>] The 1st arg is a list) (<entity_x:1>[<function>] The 1st arg is an instance of the entity) (<entity_x:n_ending_in_>[<function>] The 1st arg is an instance of the entity)	<field or entity>[<function>:<*arg1>:<*...>:<*argn>][<filter>]=<value> (args are literals. If we want references to other fields or entities, it must be in a V3.<gt;_0>)
String or Array/List length	<field or entity_x:n>[length][<filter>]=<value> Eg. countries[length][gt]=1 Eg. description[length][lte]=256 Eg. related_products[length][eq]=0
Full days passed from the field's date to the current date.	<field>[days_passed:<*tz_database_name>][<filter>]

Functions

(Since the date fields are considered to be UTC, the current date is the UTC one too, unless a tz database code)

Eg. `creation_date[days_passed][gt]=7` (UTC is inferred)
Eg. `last_update[days_passed:Europe/Madrid][lt]=1`

Custom filters (can be injected in each API)

Eg. `.[last_updated_by][eq]=admin`
Eg. `related_products.[last_updated_by][eq]=admin`
Eg. `country_code[continent]=africa`

Filters

Filters only apply to fields

Field = 'value'

field=value
(alt) field[eq]=value

(Universal Group - equality filter)
`field <> 'value' OR field IS NULL OR NOT EXISTS(field)`

(If the field is undefinable)
`_or[#]field[!eq]=value&_or[#]field[!exists]`

(If the field is nullable)
`_or[#]field[!eq]=value&_or[#]field[!null]`

(If the field is undefinable and nullable)
`_or[#]field[!eq]=value&_or[#]field[!exists]&_or[#]field[!null]`

("not equal" among the elements with value)
`field <> 'value'`

field[!eq]=value

(If the field is undefinable)
`field[!eq]=value&field[exists]`

(If the field is nullable)
`field[!eq]=value&field[!null]`

(If the field is undefinable and nullable)
`field[!eq]=value&field[exists]&field[!null]`

Field is not null

field[!null]

Field is null

field[null]

Field = '' OR Field = []

field[len]=0

NOT (Field = '' OR Field = []) AND EXISTS(Field)

field[len][gt]=0

IS UNDEFINED
(field not exists)

field[!exists]

Filters	
IS NOT UNDEFINED	field[exists]
Field like '%value%' (IGNORE CASE)	field[contains:ignorecase]=value
Field like '%value%' (CASE SENSITIVE)	field[contains]=value
With ARRAYS: ARRAY[field] && ARRAY[<value1>, <value2>] With jsonb arrays: '["a", "b", "c"]'::jsonb ?& array['a', 'b']	field[intersects]=value field[!intersects]=value
Field in ('value1', 'value2')	field[in]=value1&field[in]=value2
	_or[#]field=value1&_or[#]field=value2
Field not in ('value1', 'value2') OR field IS NULL	(If the field is undefinable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!exists] (If the field is nullable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!null] (If the field is undefinable and nullable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!exists]&_or[#]field[!null]
Field not in ('value1', 'value2')	field[!in]=value1&field[!in]=value2
Field > #	field[gt]=#
Field >= #	field[gte]=#
Field < #	field[lte]=#
Field <= #	field[lte]=#
Generic Length Comparison LEN(field) <# comparison> = value	<field>[length][<eq eq lte lte gt gte>]=<value>
Field1 like '%value%' or Field2 like '%value%' (IGNORE CASE)	_or[#]field1[icontains]=value&_or[#]field2[icontains]=value
field BETWEEN #1 AND #2	field[gte]=#1&field[lte]=#2

Filters	
Generic Days passed Comparison UTCDATE(Field) <# comparison> (UTCDATE(NOW(timezone)) - DAYS(#))	<field>[<days_passed_function>][<eq eq lt lte gt gte>]=<value>

Related entities filtering transversal		
	Current	Possible Alternative
Basic entity transversal	Not possible	related-entity.attribute[<filter>]=value (x:m related entities are named "<singular_entity_name>_set")
Custom attributes	{ "operator": "<filter>", "field": "attributes.name", "value": "value" }	attributes.name[<filter>]=value
Product relationships	"relationship_filters": [[{ "product_ids": [{ "id": "<product_id>", "qty_operator": "<op2>", "value": # }], "relationship_id": "<relationship_id>", "operator": "<op1>" }],	product_relationships.relationship_id=<relationship_id>& product_relationships.related_product_id=<product_id>& product_relationships.related_product.quantity[<op2>]=#
	Eg1 [[{ "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "exists", "product_ids": [], }]]	Eg1 product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc
	Eg2 [[{ "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "!exists", "product_ids": [], }]]	Eg2 _!exists[0]product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc For the public PIM API V3.0, it will only be allowed for relationships.id or relationships.label and without being grouped with _and or _or

Related entities filtering transversal

		relationships[have]id=5d89c6c696c8f4a322df4bbc relationships[have]label[!eq]='pack' relationships.id[have]5d89c6c696c8f4a322df4bbc
		_alias[relationships]=alias& .alias.id=5d89c6c696c8f4a322df4bbc& .alias[!len]=0
		_fields=relationships.id& relationships[!contains]={"id": "5d89c6c696c8f4a322df4bbc"}
		relationships[contains_kv:label:pack][eq]=false
	Eg3 [[{ "product_ids": [{ "id": "5d89c68596c8f4a322df4bb9", "qty_operator": "exists", "value": null}], "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "exists" }]}	product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc& product_relationships.related_product_id=5d89c68596c8f4a322df4bb9
	Eg4 [[{ "product_ids": [{ "id": "5d89c67796c8f4a322df4bb8", "qty_operator": "gt", "value": [2]}], "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "exists" }]}	product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc& product_relationships.related_product_id=5d89c67796c8f4a322df4bb8& product_relationships.related_products.quantity[gt]=2 product_relationships id_product id_relation linked_to quantity (Ambiguous: The link could belong to a different relationship) relationships.label=packed_together& relationships.links.product_id=5d89c67796c8f4a322df4bb8& relationships.links.quantity[gt]=2 Aliases might solve this
	Eg5 [[{ "product_ids": [{	product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc & product_relationships.related_product_id=5d89c67796c8f4a322df4

Related entities filtering transversal

	<pre> "id": "5d89c67796c8f4a322df4bb8", "qty_operator": "bte", "value": [1, 2] }, "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "exists"] </pre>	bb8& product_relationships.relrelated_products.quantity[lte]=2& product_relationships.relrelated_products.quantity[gte]=1 (Ambiguous: The link could belong to a different relationship) relationships.id=5d89c6c696c8f4a322df4bbc& relationships.links.id=5d89c67796c8f4a322df4bb8& relationships.links.quantity[gte]=1 & relationships.links.quantity[lte]=2 Aliases might solve this
--	--	--

Returned first level fields

	Current	Possible Alternative
Selected ones	<pre> { "attributes": [] } </pre>	_fields=<field1>&_fields<field2> (Optional)
All	Not possible, they have to be enumerated	Default behaviour _fields=*

Returned multilevel fields

	Current	Possible Alternative
Selected ones	<pre> { "attributes": [] } </pre>	_fields=<expansion1>.<field1>& [*...]&_fields=<expansion1>.<fieldn>& [*...]&_fields=<expansionn>.<field1>& [*...]&_fields=<expansionn>.<fieldn> If the <field> is *, then all fields are returned (default behaviour for ".")
All	Not possible when filtering, they have to be enumerated. Default behaviour on get by id.	_fields=<entity>.*

Related entities data expansion		
	Current	Possible Alternative
Expand all	<pre>{ "attributes": [] }</pre>	Default behaviour _expand=<m:m_relationship>& _expand=<relationship_l1>.<relationship_l2>
		_include_unfiltered_entities=<subentity> (one per subentity) (required if we want _field=<relationship> to return that field for all elements even unfiltered ones)
Expand filtered	Not possible	default behaviour using _field=<subentities>.<field>
		_expand_filtered=<relationship> (Alternative when there are already other filters on the same relationship that are only intended to indicate what 1st level entity results are retrieved) _alias[<relationship>]=<alias>& _expand=.<alias>& .<alias>.<field>[<filter>]=<value> Not discarded but it is excluded for 3.0 version relationships.label=pack& _expand=relationships relationships.label=pack& _alias[relationships]=alias& _field=.alias.links.quantity& .alias.label[len][gt]=10

`_fields=*` and `GET` should return the same result:

- `_ids` fields hided for now, as they expose denormalization (excluded from responses and for operation use)
- `_id` fields are exposed as they are foreign keys correctly modeled

- `_fields=related_entity` must match `_fields=related_entity.*` so it matches the behaviour of `_fields=json-field | _fields=json-field.*` and `GET /entity | GET /entity?_fields=*`

Pagination	
Current	Possible Alternative
<pre>"pagination": { "page": #, "page_size": #, "order": "field" }</pre>	<pre>_page=#&_page_size=#&_sort_by=<field> &_filtered_count=<bool>&_total_count=<bool></pre>

URL S STANDARDIZATION

V3			
Action	HTTP VERB	PATH	Notes
Search	GET	api/v3/<plural_entity_name>[?<search_query>] No <code>search_query</code> means "get all", unless there is an HTTP payload indicating its own filters	Response: 200 <pre>{ "data": [*<json_object_1>, ..., *<json_object_n>], "pagination": { **"next_page": <url>, **"previous_page": <url>, **"filtered_count": #, **"total_count": # } }</pre> 4xx/5xx <pre>{ "error": { "name": "<name>", "description": "<description>" } }</pre>

Get one entity instance	GET	<code>api/v3/<plural_entity_name>/<identifier>?<*expansions>*<fields_selection></code> <code>api/v3/<plural_entity_name>/<identifier><subentities>?<*expansions>*<fields_selection></code> <code><subentities> = (</subentity>)+</code> <code><subentity> = <x:m related entity> <x:1 related entity></code> <code><x:m related entity> = <plural_entity_name>/<identifier></code> <code><x:1 related entity> = <entity_name></code>	Identifiers will usually be the primary key field of the entity. For some entities, the identifiers can be other fields. They require dto be fields that are unique for one account<>entity. Eg. For products, both id and sku (correctly url-encoded) can be used as identifiers.
Create	POST	<code>api/v3/<plural_entity_name></code>	“EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM RETURN 201 { id: }
Override selected fields of one entity instance	PATCH	<code>api/v3/<plural_entity_name>/<identifier></code> <code>api/v3/<plural_entity_name>/<identifier><subentities></code>	“EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM RETURN 204 OK
Delete one entity instance	DELETE	<code>api/v3/<plural_entity_name>/<identifier></code> <code>api/v3/<plural_entity_name>/<identifier><subentities></code>	RETURN 204 OK
Get one entity instance field	GET	<code>api/v3/<plural_entity_name>/<identifier>/<field></code> <code>api/v3/<plural_entity_name>/<identifier><subentities>/<field></code>	
Create an entity instance field	POST	<code>api/v3/<plural_entity_name>/<identifier>/<field></code> <code>api/v3/<plural_entity_name>/<identifier><subentities>/<field></code>	It will only work on attributes “EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM
Delete an entity instance field	DELETE	<code>api/v3/<plural_entity_name>/<identifier>/<field></code> <code>api/v3/<plural_entity_name>/<identifier><subentities>/<field></code>	It will only work on attributes
Advanced updates	PATCH	<code>api/v3/<plural_entity_name>/<identifier></code> <code>api/v3/<plural_entity_name>/<identifier><subentities></code>	Indicated by adding the content type: application/json-patch+json https://datatracker.ietf.org/doc/html/rfc6901 https://datatracker.ietf.org/doc/html/rfc6902
1:M Link a new entity instance	POST	<code>api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name></code>	<pre>{ "id": <related_entity_id>, [<M:M stuff>] }</pre>
1:M get a association entity instance	GET	<code>api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id></code>	

1:M Edit a Relationship entity instance	PATCH	api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id>	{ "id": <related_entity_id>, [<M:M stuff>] }
1:M Unlink a new entity instance	DELETE	api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id>	It deletes all the coincidences in the strange case of M:M relationships that allow repetitions.
Bulk additions	POST	api/v3/-/bulk/<plural_entity_name> List of items equal to the non bulk POST Payload: { *"webhook": <url>, *"external_reference": <str>, "data": [{}, {} ... {}] }	Response: { "bulk_job_id": <id> }
Bulk edits (a filter and the same edit for all matches)	PATCH	api/v3/-/bulk/<plural_entity_name>/?<search_query> Same payload as the non bulk PATCH Payload: { *"webhook": <url>, *"external_reference": <str>, "data": {"<field_1>": <value_1>, ..., "<field_n>": <value_n>} }	"EMPTY" VALUES SUCH AS "" OR [] ARE STORED VERBATIM AND DON'T MEAN DELETION
Bulk edits (different edit per entity instance)	PATCH	api/v3/-/bulk/<plural_entity_name>/ List of items equal to the non bulk PATCH plus the id Payload: { *"webhook": <url>, *"external_reference": <str>, "data": [{"id": <id_1>, ...}, ..., {"id": <id_n>, ...}] }	"EMPTY" VALUES SUCH AS "" OR [] ARE STORED VERBATIM AND DON'T MEAN DELETION
Bulk deletes by filter	DELETE	api/v3/-/bulk/<plural_entity_name>/?<search_query> { *"webhook": <url>, *"external_reference": <str> }	
Bulk deletes by individual items	DELETE	api/v3/-/bulk/<plural_entity_name>/ Payload:	

		<pre>{ *webhook*: <url>, *external_reference*: <str>, "data": [id, id, id] }</pre>	
Advanced bulk updates	PATCH	api/v3/-/bulk/-/edit/<plural_entity_name>/?<search_query> The payload format is the same one of the advanced updates	
Bulk Individual updates	POST	api/v3/-/bulk/-/edit The payload contains a list of individual actions (deletions, creations and updates) <pre>{ *webhook*: <url>, *external_reference*: <str>, "data": [{ "entity": 'products', "action": "PATCH DELETE POST", data: { individual-object }}} }</pre> The result, except for performance, is equivalent to individually calling the other update endpoints one by one.	
Bulks: check job	GET	api/v3/-/bulk/-/jobs/<bulk_job_id>	Response: <pre>{ "state": <str>, "webhook": <url none>, "external_reference": <str none>, "created": <datetime>, "updated": <datetime>, "data": <object none> }</pre>
Bulks: search job	GET	api/v3/-/bulk/-/jobs/?<search_query> queryable attributes: external_reference <str> state <str>	
Bulks: transition job	POST	api/v3/-/bulk/-/jobs/<bulk_job_id>/<cancel pause resume retry> api/v3/-/bulk/-/jobs/<bulk_job_id>/-/<cancel pause resume retry>	
Ad-Hoc async action (eg. Shopify export)	POST	api/v3/-/<action-name>/ <pre>{ *webhook*: <url>, *external_reference*: <str>, </pre>	Response: <pre>{ "action-name_job_id": <id> }</pre>

		<pre>"data": "<whatever>" }</pre> eg. api/v3/-/shopify-export/	<pre>eg. { "shopify-export_job_id": <id> }</pre>
Ad-Hoc async action: check job	GET	<pre>api/v3/-/<action-name>/-/jobs/<action-name_job_id></pre>	Response: <pre>{ "state": <str>, "webhook": <url none>, "external_reference": <str none>, "created": <datetime>, "updated": <datetime>, "data": <object none> }</pre>
Ad-Hoc async action: search jobs	GET	<pre>GET api/v3/-/<action-name>/-/jobs/?<search_query></pre> queryable attributes: external_reference <str>	
Ad-Hoc async action: transition job	POST	<pre>api/v3/-/bulk/-/<action-name>/<action-name_job_id>/<cancel pause resume retry></pre> <pre>api/v3/-/<action-name>/-/jobs/<action-name_job_id>/-/<cancel pause resume retry></pre>	

200 OK → SYNC
201 OK → SYNC → ID
202 JOB → ASYNC

Response Conventions for Entity Modification Endpoints

General

For the definition of the new API, the following conventions will apply:

- **POST** requests for creating entities, **PUT** requests for updating entities, and **PATCH** requests will **not** return the modified entity in the response.
- Instead:
 - A **201 Created** status will be returned if the creation or modification has been successfully completed.

[201 Created](#)

The request succeeded, and a new resource was created as a result. This is typically the response sent after [POST](#) requests, or some [PUT](#) requests.

- A **202 Accepted** status will be returned if the request has been accepted but the processing is still ongoing.

[202 Accepted](#)

The request has been received but not yet acted upon. It is noncommittal, since there is no way in HTTP to later send an asynchronous response indicating the outcome of the request. It is intended for cases where another process or server handles the request, or for batch processing.

- In the case of **201 Created**, the response body will include the **ID of the created or modified entity**.
- In the case of **202 Accepted**, the response body will include a **process_id** that can be used to track the status of the ongoing operation.

PATCH

Instead, the recommended approach is to use **PATCH** requests, where only the intended changes are explicitly specified. This method provides more precise control over updates, reduces the risk of accidental data loss, and simplifies concurrency management. Adopting PATCH as the standard for modifications ensures a safer and more maintainable API design.

DELETE

In API V3 a **DELETE** operation erases the entity or field (if using a path based **DELETE**) it is performed over, it doesn't set null or applies defaults (it can cause inheritance to resolve for the field)

ANEXO

Carácter	Significado en URL
=	separa clave/valor
&	separa parámetros
?	inicia la query
#	inicia fragmento
+	puede interpretarse como espacio
,	a veces reservado por proxies antiguos
< >	no válidos en URL (se deben escapar)
"	pueden romper encoding
%	indica secuencia de escape
()	válidos pero deberían codificarse según RFC 3986
(espacio)	se convierte en + o %20

PROCESS MANAGER API

URL S STANDARDIZATION

	V3		
Action	HTTP VERB	PATH	Notes
Search, get, patch, post, delete	ALL	ALL CRUD OPERATIONS EXPOSED WITH API V3	For processes, we won't let PATCHes for their statuses (process transitions will be handled in ad-hoc endpoints)
	POST	/api/vx/<to_be_decided>/	INPUT PAYLOAD: <pre>{ "external_id": <optional>, "webhook": <optional>, "payload": { process_type: ... input_data: } }</pre>

			RESPONSE: { " } }
--	--	--	-------------------------------